

Lab Report No13
Introduction to Filters in Digital Signal Processing
(DSP): FIR and IIR
Digital Signal Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

Submitted to:

Engr. Hareem Khan

Dated:

31th May, 2024

Department of Computer Engineering,
HITEC University, Taxila

Lab Example No 1:

Solution:

Brief description (3-5 lines)
An FIR filter is designed by finding the coefficients and filter order that meet certain specifications, which can be in the time-domain (e.g. a matched filter) and/or the frequency domain (most common). Matched filters perform a cross-correlation between the input signal and a known pulse-shape. The FIR convolution is a cross-correlation between the input signal and a time-reversed copy of the impulse-response. Therefore, the matched-filter's impulse response is "designed" by sampling the known pulse-shape and using those samples in reverse order as the coefficients of the filter.
The code

```

fir.py > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.signal import firwin, freqz
4  wp = 0.375 * np.pi
5  ws = 0.5 * np.pi
6  wt = ws - wp
7  n1 = np.ceil(8 * np.pi / wt)
8  N = int(n1 + (n1 - 1) % 2)
9  print('Order of the FIR filter N =', N)
10 Wc1 = wp + wt / 2
11 fs = 2 * np.pi
12 Wc = Wc1 / (0.5 * fs)
13 print('Cutoff frequency =', Wc)
14 h = firwin(N, Wc, window='hamming')
15 print('Impulse Response of FIR filter:')
16 print(h)
17 plt.figure(1)
18 w, H = freqz(h)
19 plt.stem(w, np.abs(H))
20 plt.title('Frequency Response')
21 plt.xlabel('Frequency (radians)')
22 plt.ylabel('Magnitude')
23 plt.grid(True)
24 plt.figure(2)
25 n = np.arange(N)
26 plt.stem(n, h)
27 plt.xlabel('n')
28 plt.ylabel('h(n)')
29 plt.title('Impulse Response of Filter')
30 plt.grid(True)
31 plt.show()

```

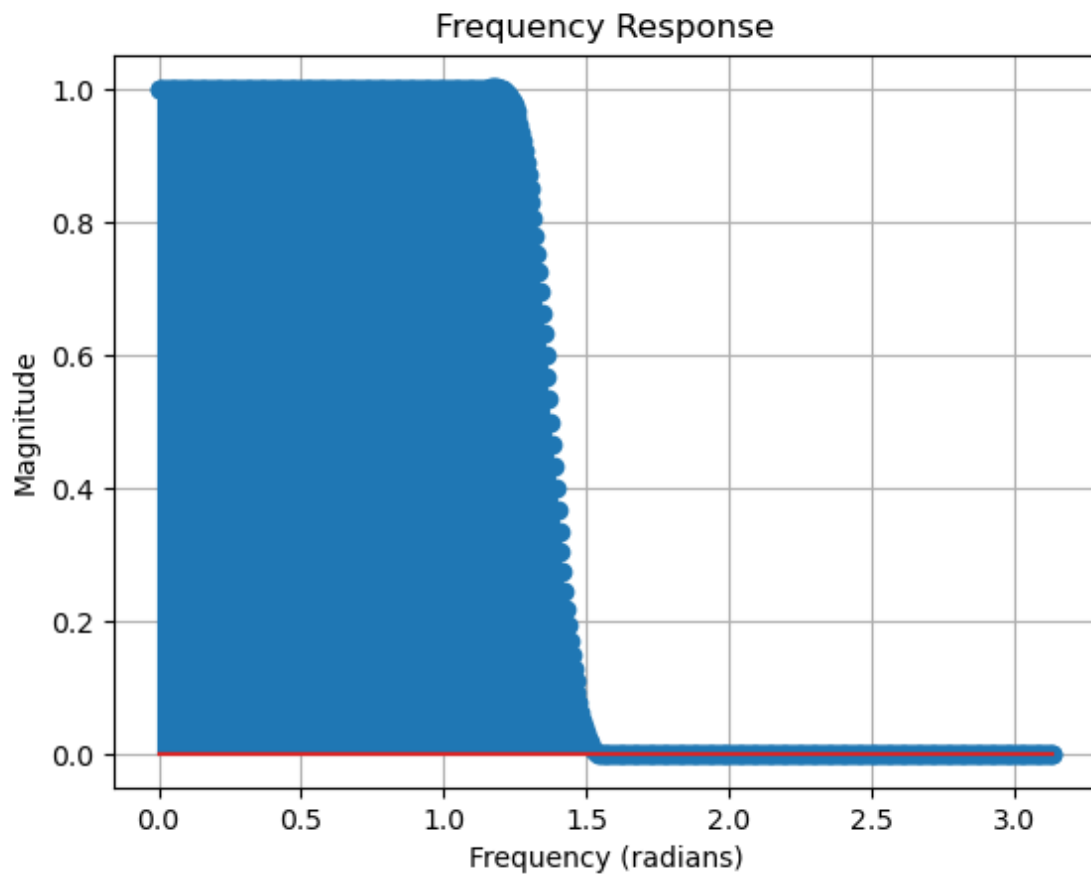
The results (Screenshot)

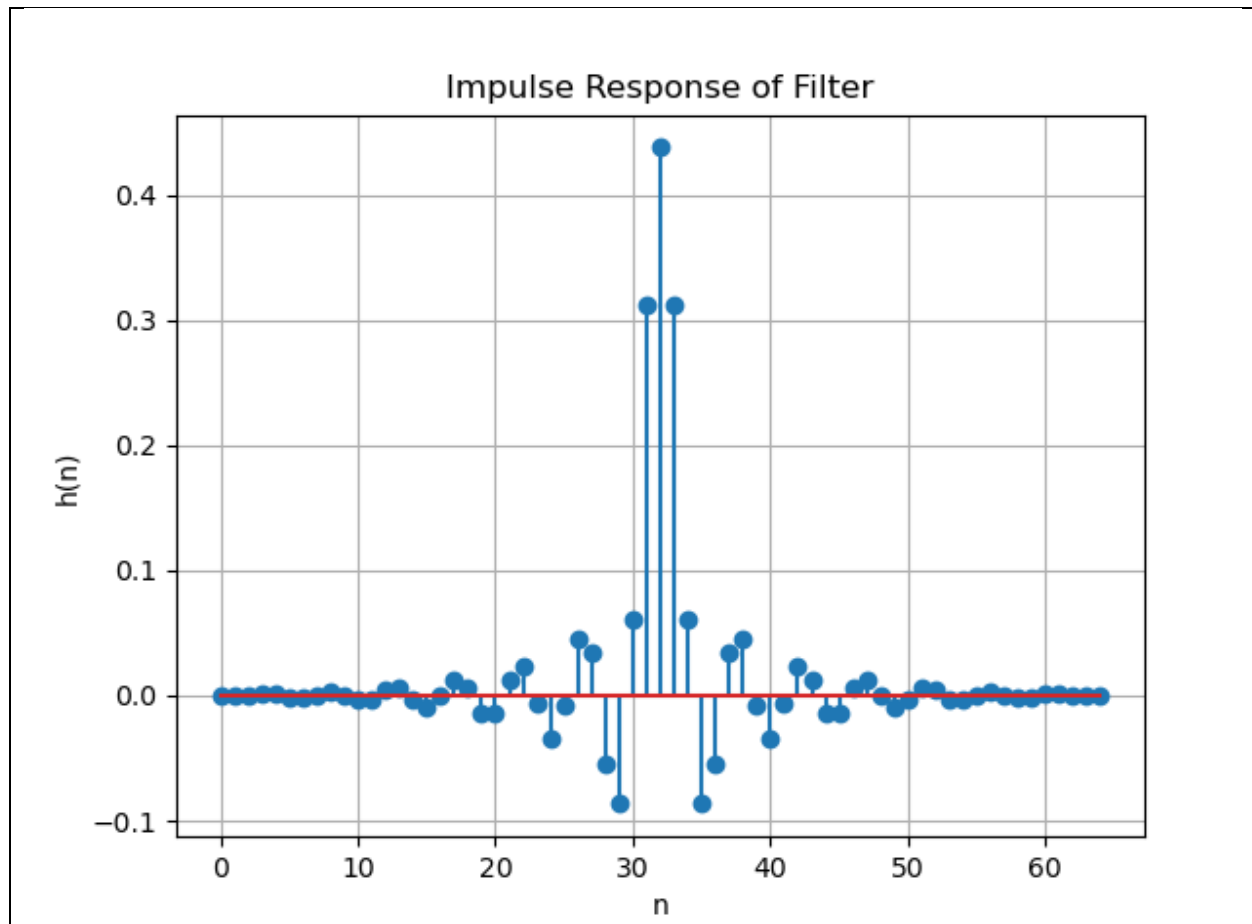
```

4.09988631e-03  5.66730655e-03  -3.04989910e-03  -9.09723779e-03
9.21805112e-18  1.21887797e-02  5.48441633e-03  -1.37251478e-02
-1.34429342e-02  1.21789049e-02  2.34177943e-02  -5.74486057e-03
-3.44601894e-02  -7.95244816e-03  4.52559436e-02  3.34789130e-02
-5.43502855e-02  -8.65551612e-02  6.04241311e-02  3.11793705e-01
4.37909506e-01  3.11793705e-01  6.04241311e-02  -8.65551612e-02
-5.43502855e-02  3.34789130e-02  4.52559436e-02  -7.95244816e-03
-3.44601894e-02  -5.74486057e-03  2.34177943e-02  1.21789049e-02
-1.34429342e-02  -1.37251478e-02  5.48441633e-03  1.21887797e-02
9.21805112e-18  -9.09723779e-03  -3.04989910e-03  5.66730655e-03
4.09988631e-03  -2.72389497e-03  -3.80571494e-03  6.70701873e-04
2.85062249e-03  4.58510044e-04  -1.78338501e-03  -8.80562332e-04
9.25421244e-04  9.11733196e-04  -3.61058348e-04  -8.28743024e-04
-1.36563720e-18]

```

GRAPH:





Lab Example No 2:

Solution:

Brief description (3-5 lines)

An FIR filter is designed by finding the coefficients and filter order that meet certain specifications, which can be in the time-domain (e.g. a matched filter) and/or the frequency domain (most common). Matched filters perform a cross-correlation between the input signal and a known pulse-shape. The FIR convolution is a cross-correlation between the input signal and a time-reversed copy of the impulse-response. Therefore, the matched-filter's impulse response is "designed" by sampling the known pulse-shape and using those samples in reverse order as the coefficients of the filter.

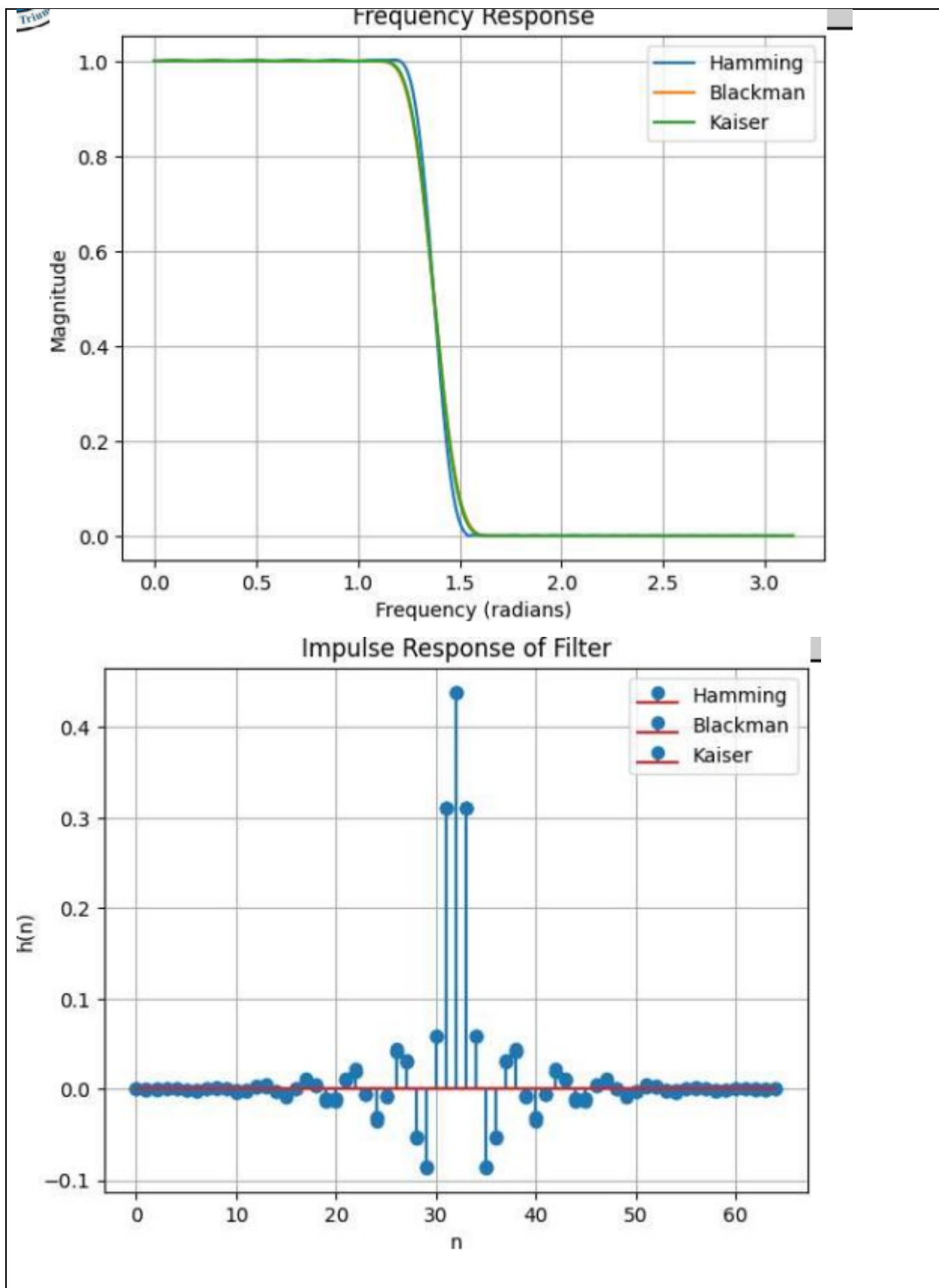
The code

```

DIFFERENTWINDOWS.PY > ...
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import firwin, freqz
4 wp = 0.375 * np.pi
5 ws = 0.5 * np.pi
6 wt = ws - wp
7 n1 = np.ceil(8 * np.pi / wt)
8 N = int(n1 + (n1 - 1) % 2)
9 print('Order of the FIR filter N =', N)
10 h_hamming = firwin(N, wc, window='hamming')
11 h_blackman = firwin(N, wc, window='blackman')
12 beta = 8
13 h_kaiser = firwin(N, wc, window=('kaiser', beta))
14 plt.figure(1)
15 w, H_hamming = freqz(h_hamming)
16 w, H_blackman = freqz(h_blackman)
17 w, H_kaiser = freqz(h_kaiser)
18 plt.plot(w, np.abs(H_hamming), label='Hamming')
19 plt.plot(w, np.abs(H_blackman), label='Blackman')
20 plt.plot(w, np.abs(H_kaiser), label='Kaiser')
21 plt.title('Frequency Response')
22 plt.xlabel('Frequency (radians)')
23 plt.ylabel('Magnitude')
24 plt.legend()
25 plt.grid(True)
26 plt.figure(2)
27 n = np.arange(N)
28 plt.stem(n, h_hamming, label='Hamming')
29 plt.stem(n, h_blackman, label='Blackman')
30 plt.stem(n, h_kaiser, label='Kaiser')
31 plt.xlabel('n')
32 plt.ylabel('h(n)')
33 plt.title('Impulse Response of Filter')
34 plt.legend()
35 plt.grid(True)
36 plt.show()

```

The results (Screenshot)



Lab Example No 3:

Solution:

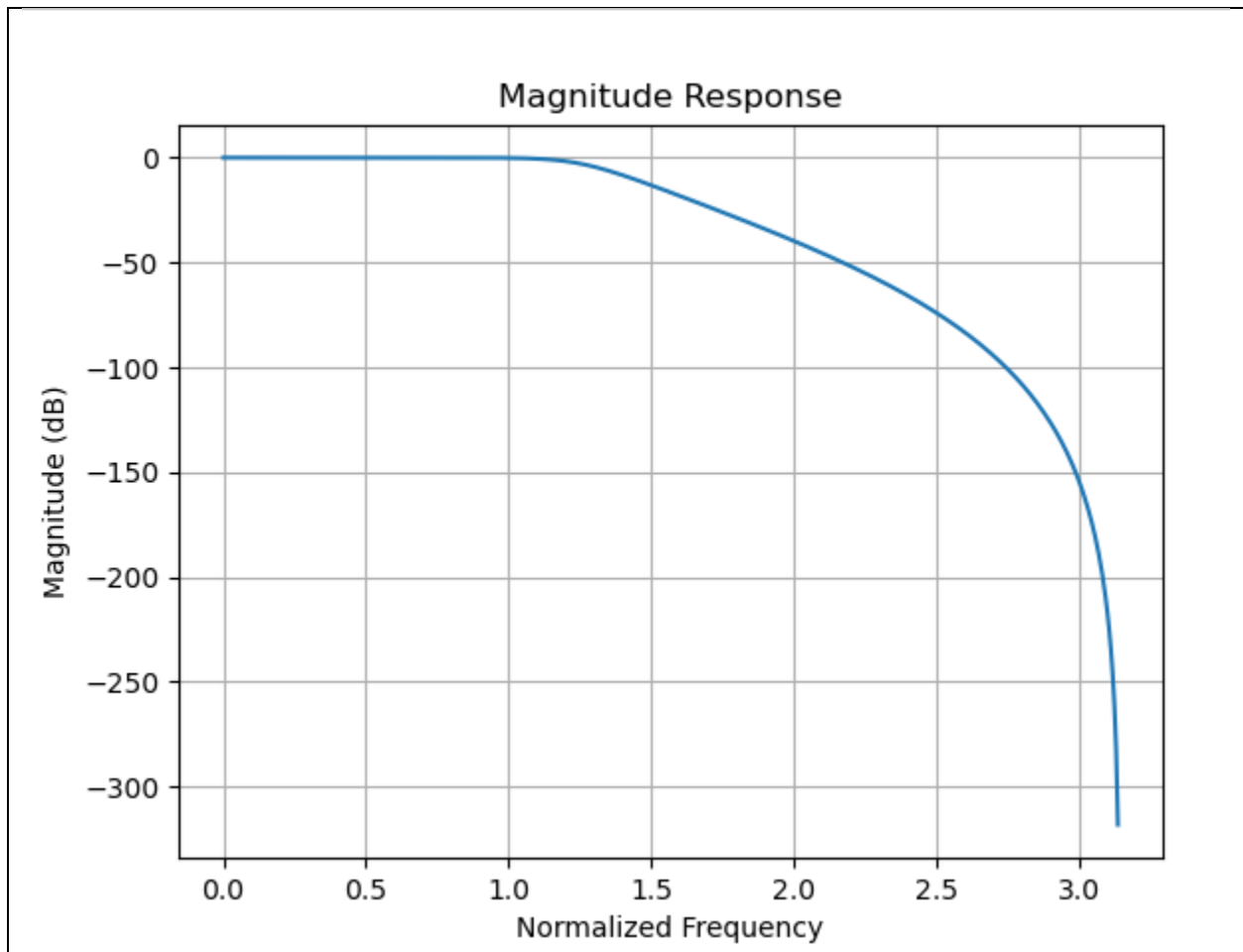
Brief description (3-5 lines)
IIR filters are digital filters with infinite impulse response. Unlike FIR filters, they have the feedback (a recursive part of a filter) and are known as recursive digital filters therefore. For this reason IIR filters have much better frequency response than FIR filters of the same order. Unlike FIR filters, their phase characteristic is not linear which can cause a problem to the systems which need phase linearity. For this reason, it is not preferable to use IIR filters in digital signal processing when the phase is of the essence. Otherwise, when the linear phase characteristic is not important, the use of IIR filters is an excellent solution.
<pre>import numpy as np import matplotlib.pyplot as plt from scipy.signal import butter, cheby1, freqz from scipy.constants import pi # Step I: Enter the pass band ripple (rp) and stop band ripple (rs) rp = 0.1 rs = 40 # Step II: Enter the pass band frequency (wp) and stop band frequency (ws) wp = 0.375 * pi ws = 0.5 * pi # Step III: Get the sampling frequency (fs) fs = 2 * pi # Step IV: Calculate normalized pass band frequency, and normalized stop band frequency w1 and w2 respectively w1 = 2 * wp / fs w2 = 2 * ws / fs # Step V: Calculate the filter order # Butterworth filter order # [n, wn] = buttord(w1, w2, rp, rs) # Chebyshev filter order # [n, wn] = cheb1ord(w1, w2, rp, rs) # Example filter order and wn values for demonstration n = 6 wn = 0.4</pre>

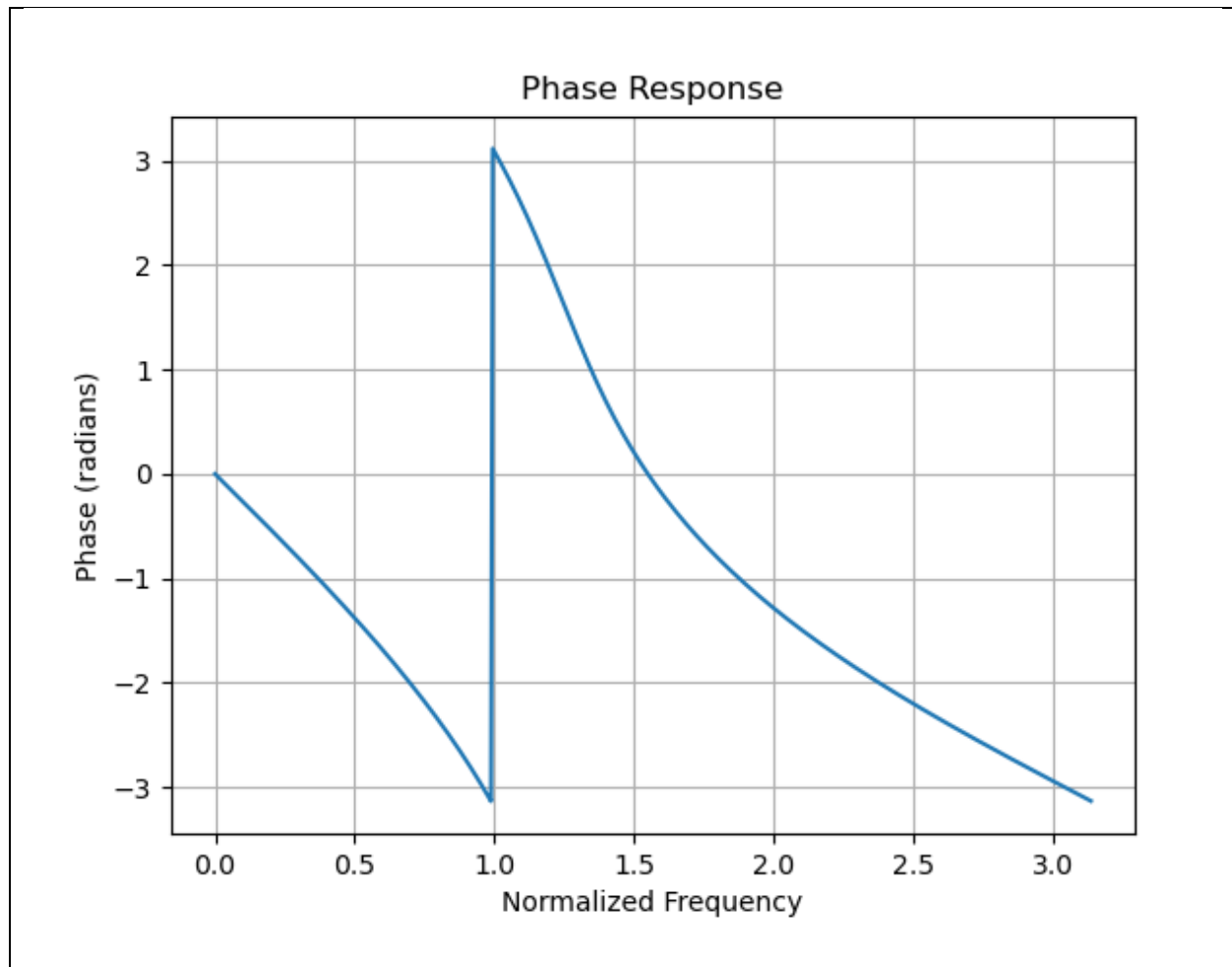

```

# Step VI: Design an nth order digital low pass Butterworth or Chebyshev filter
# Butterworth filter
b, a = butter(n, wn)
# Chebyshev filter
# b, a = cheby1(n, rp, wn)
# Step VII: Find the digital frequency response of the filter
w, H = freqz(b, a)
# Step VIII: Calculate the magnitude of the frequency response in decibels (dB)
mag = 20 * np.log10(abs(H))
# Step IX: Plot the magnitude response [magnitude in dB Vs normalized
frequency]
plt.figure(1)
plt.plot(w, mag)
plt.title('Magnitude Response')
plt.xlabel('Normalized Frequency')
plt.ylabel('Magnitude (dB)')
plt.grid(True)
# Step X: Calculate the phase response using angle(H)
phase = np.angle(H)
# Step XI: Plot the phase response [phase in radians Vs normalized frequency
(Hz)]
plt.figure(2)
plt.plot(w, phase)
plt.title('Phase Response')
plt.xlabel('Normalized Frequency')
plt.ylabel('Phase (radians)')
plt.grid(True)
plt.show()

```

The results (Screenshot)





Lab Example No 4:

Solution:

Brief description (3-5 lines)
IIR filters are digital filters with infinite impulse response. Unlike FIR filters, they have the feedback (a recursive part of a filter) and are known as recursive digital filters therefore. For this reason IIR filters have much better frequency response than FIR filters of the same order. Unlike FIR filters, their phase characteristic is not linear which can cause a problem to the systems which need phase linearity. For this reason, it is not preferable to use IIR filters in digital signal processing when the phase is of the essence. Otherwise, when the linear phase characteristic is not important, the use of IIR filters is an excellent solution.
The code

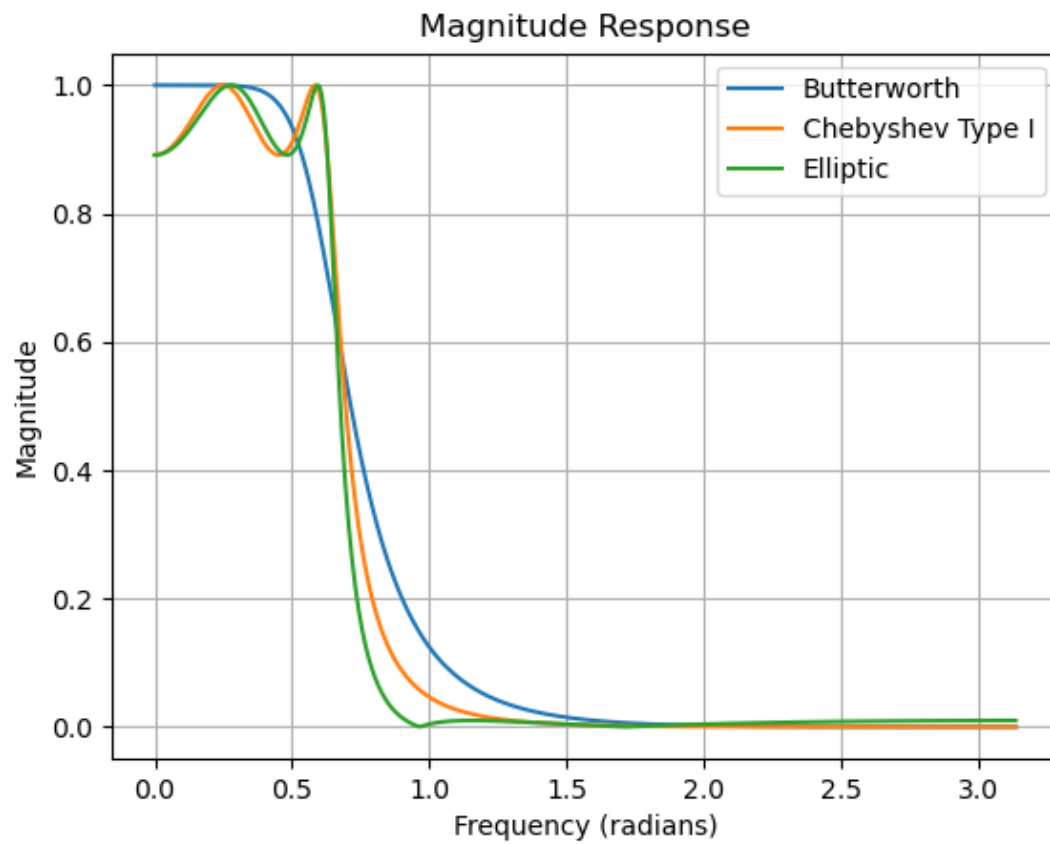
```

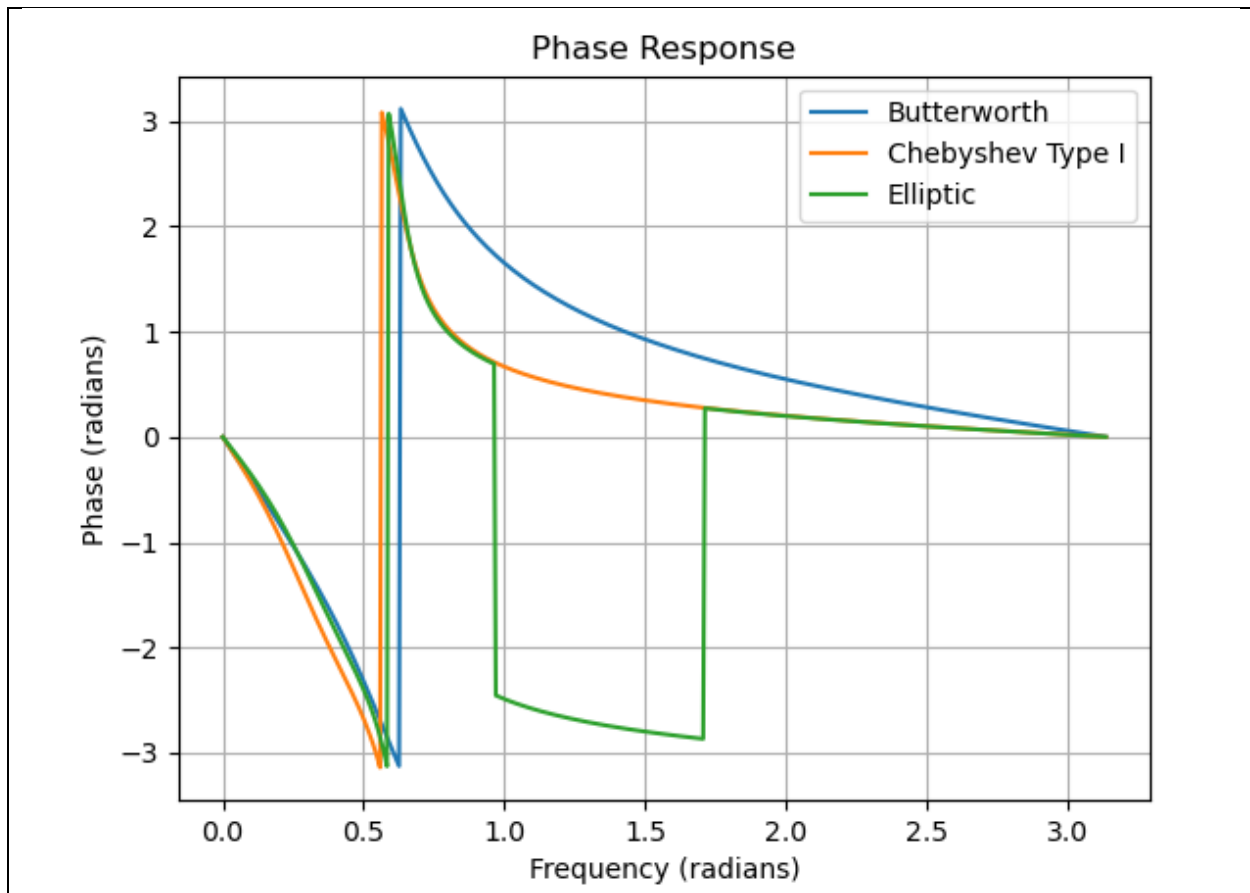
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, cheby1, ellip, freqz
# Filter specifications
order = 4
cutoff_freq = 0.2 # Normalized cutoff frequency
# Design the Butterworth filter
b_butter, a_butter = butter(order, cutoff_freq, btype='low', analog=False,
output='ba')
# Design the Chebyshev Type I filter
rp_cheby1 = 1 # Passband ripple in dB
b_cheby1, a_cheby1 = cheby1(order, rp_cheby1, cutoff_freq, btype='low',
analog=False, output='ba')
# Design the elliptic filter
rp_ellip = 1 # Passband ripple in dB
rs_ellip = 40 # Stopband attenuation in dB
b_ellip, a_ellip = ellip(order, rp_ellip, rs_ellip, cutoff_freq, btype='low',
analog=False, output='ba')
# Compute the frequency response for each filter
w, h_butter = freqz(b_butter, a_butter)
w, h_cheby1 = freqz(b_cheby1, a_cheby1)
w, h_ellip = freqz(b_ellip, a_ellip)
# Plot the magnitude response for each filter
plt.figure()
plt.plot(w, np.abs(h_butter), label='Butterworth')
plt.plot(w, np.abs(h_cheby1), label='Chebyshev Type I')
plt.plot(w, np.abs(h_ellip), label='Elliptic')
plt.title('Magnitude Response')
plt.xlabel('Frequency (radians)')
plt.ylabel('Magnitude')
plt.legend()
plt.grid(True)
# Plot the phase response for each filter
plt.figure()
plt.plot(w, np.angle(h_butter), label='Butterworth')
plt.plot(w, np.angle(h_cheby1), label='Chebyshev Type I')
plt.plot(w, np.angle(h_ellip), label='Elliptic')
plt.title('Phase Response')
plt.xlabel('Frequency (radians)')
plt.ylabel('Phase (radians)')

```

```
plt.legend()  
plt.grid(True)  
plt.show()
```

The results (Screenshot)





Lab Task No 1:

Solution:

Brief description (3-5 lines)

Hamming Window

Write a Python program to design an FIR filter using the Hamming window. Perform the following steps:

Define the input specifications: $w_p = 0.375 * \pi$, $w_s = 0.5 * \pi$.

Calculate the order of the filter using the formula $N = \text{int}(\text{np.ceil}(8 * \pi / (w_s - w_p)) + 1)$.

Calculate the cutoff frequency: $W_c = (w_p + (w_s - w_p) / 2) / (0.5 * \pi)$.

Calculate the filter coefficients using the Hamming window:

`h_hamming = firwin(N, Wc, window='hamming')`.

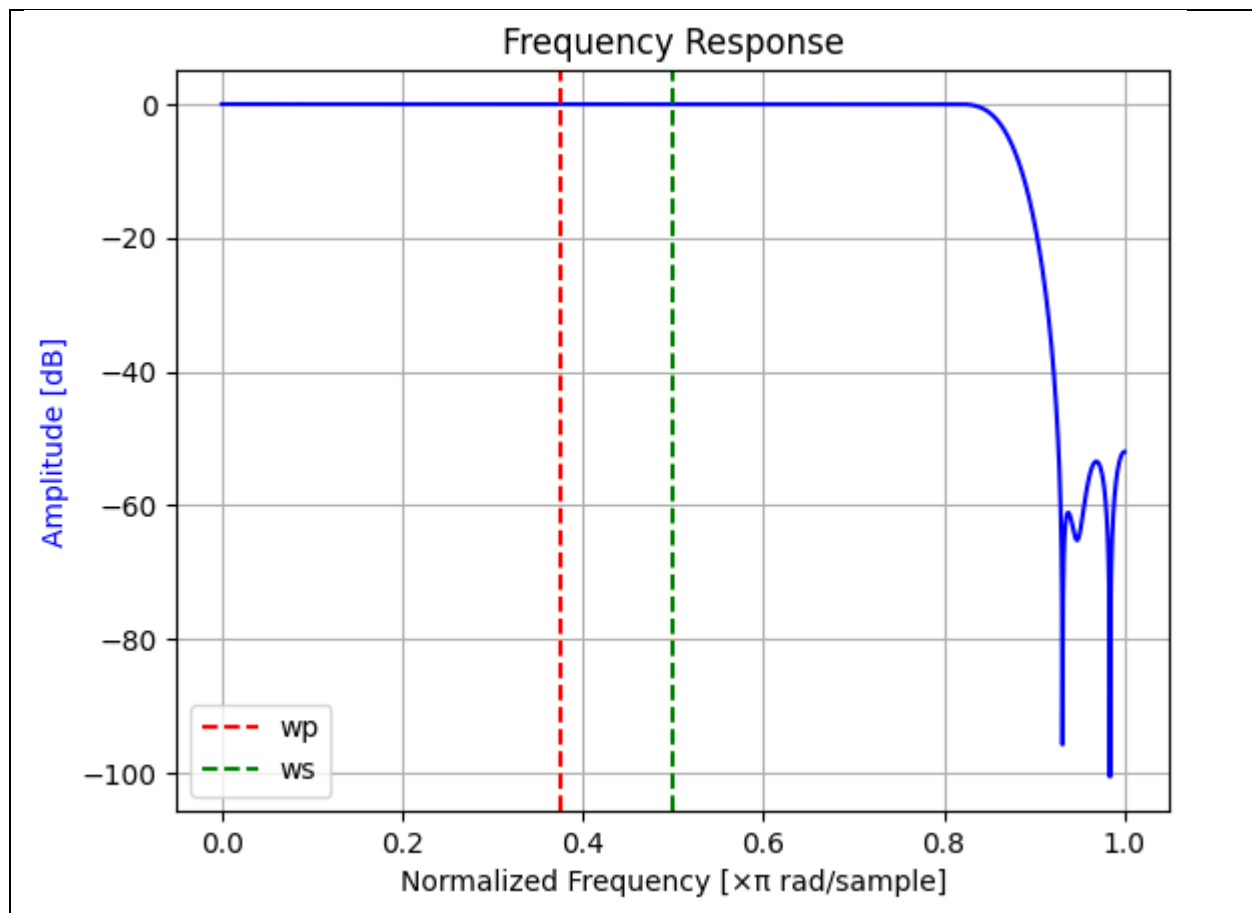
Plot the frequency response and impulse response for the Hamming window.

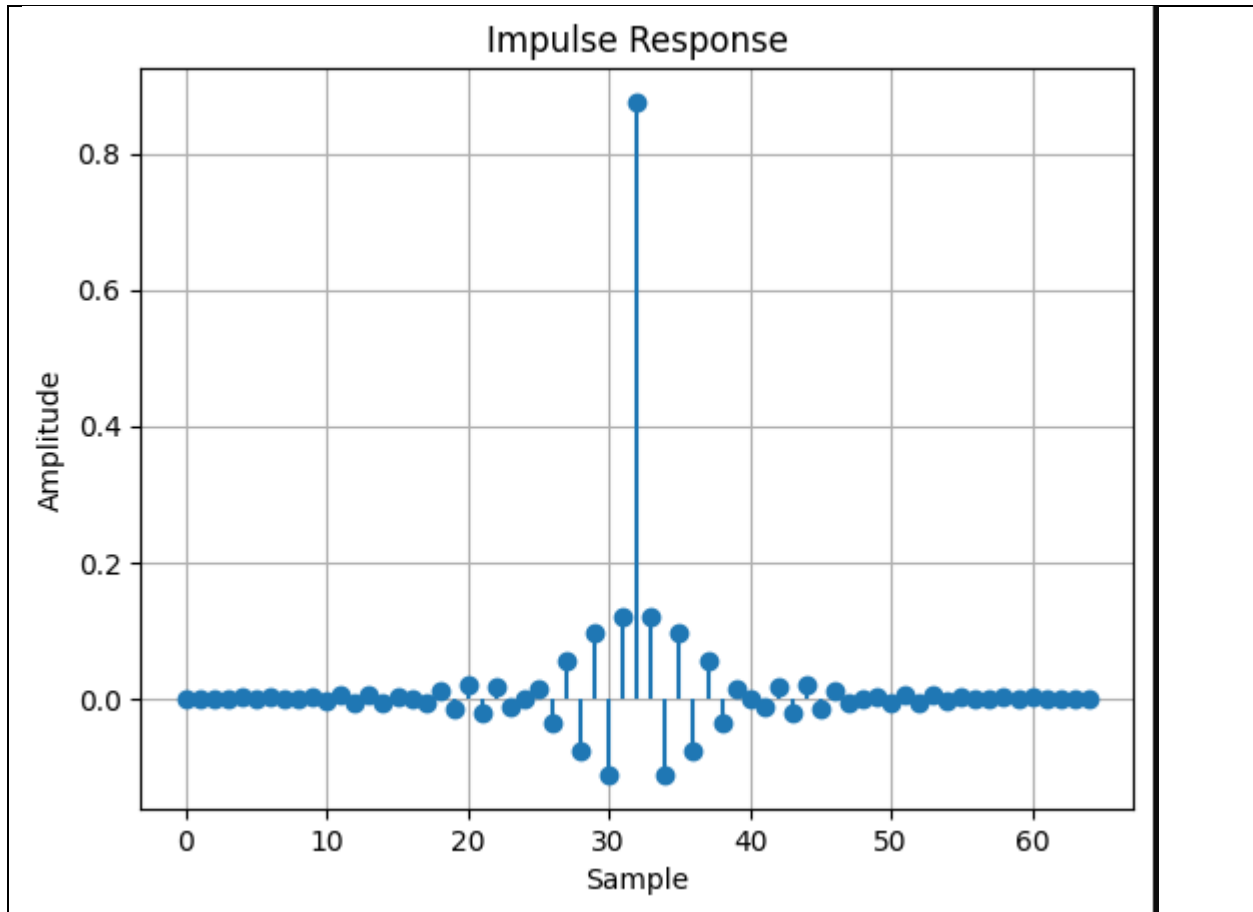
Display the plots.

The code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import firwin, freqz
4 # Input specifications
5 wp = 0.375 * np.pi
6 ws = 0.5 * np.pi
7 # Calculate the order of the filter
8 N = int(np.ceil(8 * np.pi / (ws - wp)) + 1)
9 # Calculate the cutoff frequency
10 Wc = (wp + (ws - wp) / 2) / (0.5 * np.pi)
11 # Calculate the filter coefficients using the Hamming window
12 h_hamming = firwin(N, Wc, window='hamming')
13 # Plot the frequency response
14 w, h = freqz(h_hamming, worN=8000)
15 plt.figure()
16 plt.title('Frequency Response')
17 plt.plot(w / np.pi, 20 * np.log10(np.abs(h)), 'b')
18 plt.ylabel('Amplitude [dB]', color='b')
19 plt.xlabel('Normalized Frequency [ $\pi$  rad/sample]')
20 plt.grid()
21 plt.axvline(wp / np.pi, color='red', linestyle='--', label='wp')
22 plt.axvline(ws / np.pi, color='green', linestyle='--', label='ws')
23 plt.legend()
24 # Plot the impulse response
25 plt.figure()
26 plt.title('Impulse Response')
27 plt.stem(np.arange(N), h_hamming, basefmt=" ", use_line_collection=True)
28 plt.xlabel('Sample')
29 plt.ylabel('Amplitude')
30 plt.grid()
31 # Show the plots
32 plt.show()
```

The results (Screenshot)





Lab Task No 2:

Solution:

Brief description (3-5 lines)

Blackman Window:

Write a Python program to design an FIR filter using the Blackman window. Perform the following steps:

Define the input specifications: $w_p = 0.375 * \pi$, $w_s = 0.5 * \pi$.
 Calculate the order of the filter using the formula $N = \text{int}(\text{np.ceil}(8 * \pi / (w_s - w_p)) + 1)$.

Calculate the cutoff frequency: $W_c = (w_p + (w_s - w_p) / 2) / (0.5 * \pi)$.

Calculate the filter coefficients using the Blackman window:

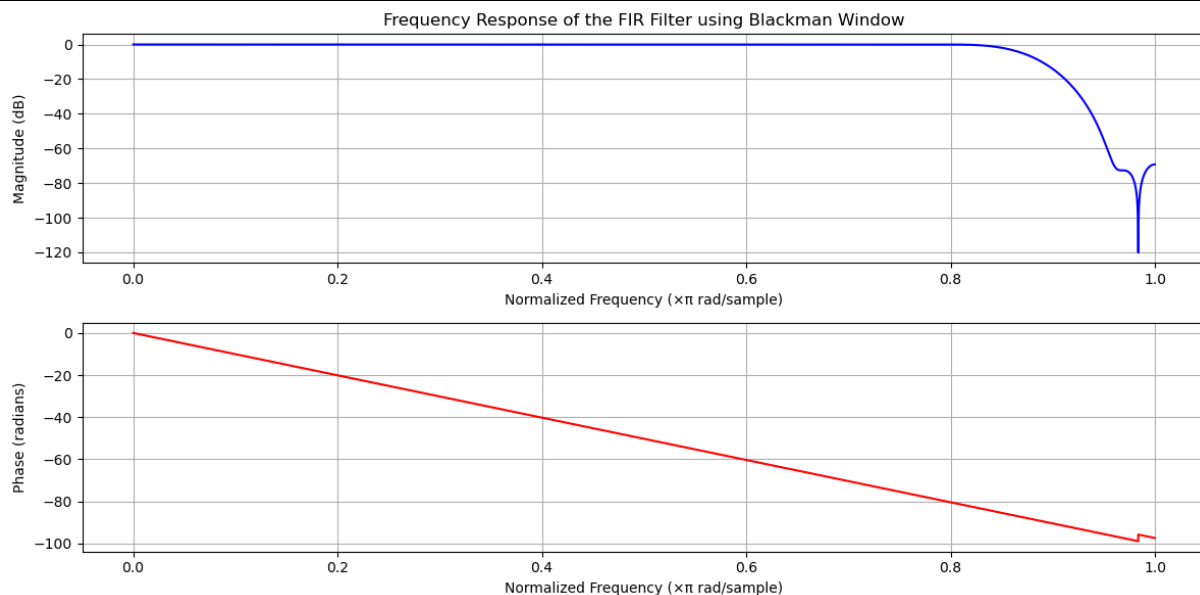
`h_blackman = firwin(N, Wc, window='blackman')`.

The code

FIRBLACKWINDOW.PY > ...

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import firwin, freqz
4 # Input specifications
5 wp = 0.375 * np.pi # Passband edge frequency
6 ws = 0.5 * np.pi   # Stopband edge frequency
7 # Calculate the filter order
8 N = int(np.ceil(8 * np.pi / (ws - wp)) + 1)
9 # Calculate the cutoff frequency
10 Wc = (wp + (ws - wp) / 2) / (0.5 * np.pi)
11 # Calculate the filter coefficients using the Blackman window
12 h_blackman = firwin(N, Wc, window='blackman')
13 # Frequency response of the filter
14 w, h = freqz(h_blackman, worN=8000)
15 # Plotting the frequency response
16 plt.figure(figsize=(12, 6))
17 plt.subplot(2, 1, 1)
18 plt.plot(w / np.pi, 20 * np.log10(np.abs(h)), 'b')
19 plt.title('Frequency Response of the FIR Filter using Blackman Window')
20 plt.xlabel('Normalized Frequency ( $\times \pi$  rad/sample)')
21 plt.ylabel('Magnitude (dB)')
22 plt.grid()
23 plt.subplot(2, 1, 2)
24 plt.plot(w / np.pi, np.unwrap(np.angle(h)), 'r')
25 plt.xlabel('Normalized Frequency ( $\times \pi$  rad/sample)')
26 plt.ylabel('Phase (radians)')
27 plt.grid()
28 plt.tight_layout()
29 plt.show()
```

The results (Screenshot)



Lab Task No 3:

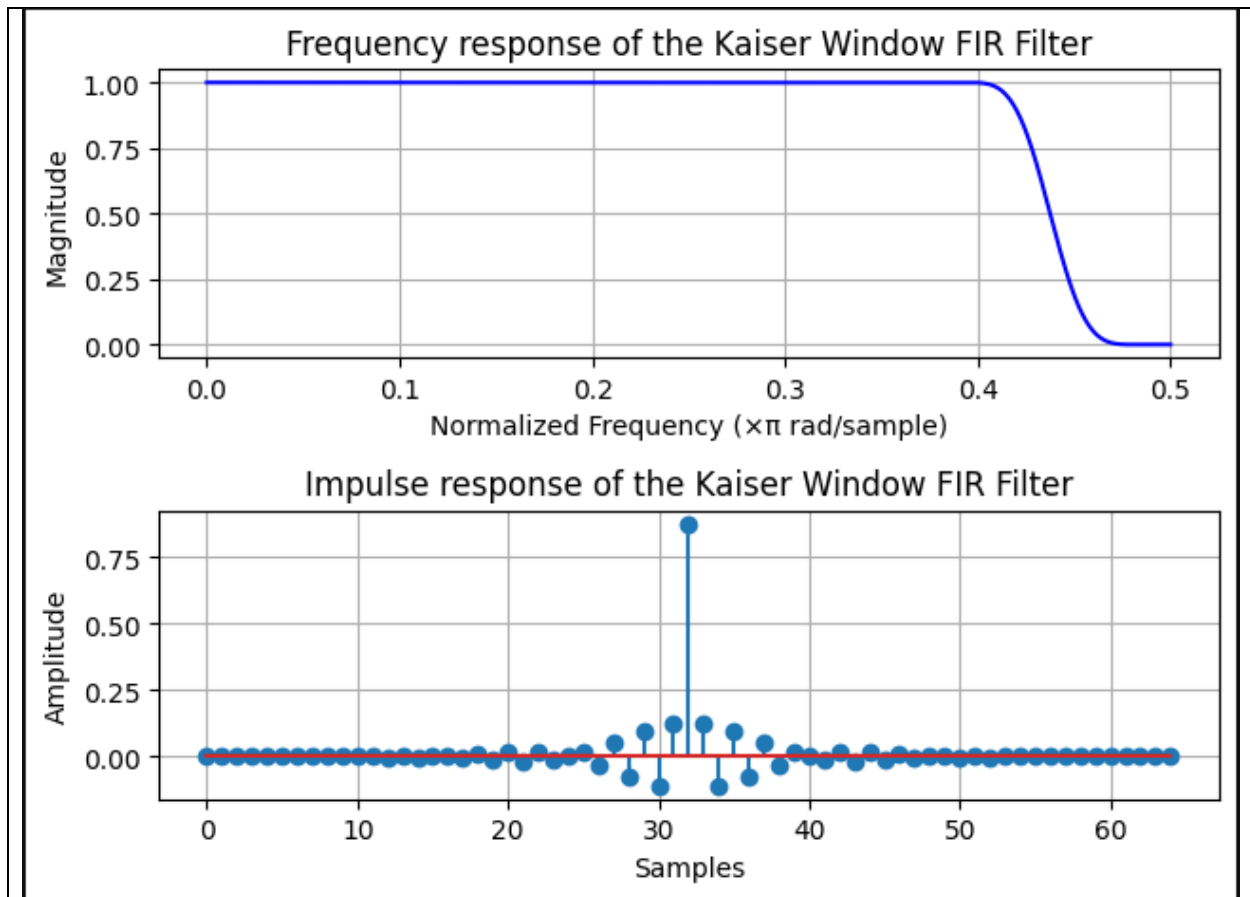
Solution:

Brief description (3-5 lines)
Kaiser Window: Write a Python program to design an FIR filter using the Kaiser window. Perform the following steps: Define the input specifications: $w_p = 0.375 * \pi$, $w_s = 0.5 * \pi$. Calculate the order of the filter using the formula $N = \text{int}(\pi * \text{ceil}(8 * \pi / (w_s - w_p)) + 1)$. Calculate the cutoff frequency: $W_c = (w_p + (w_s - w_p) / 2) / (0.5 * \pi)$. Set the beta parameter for the Kaiser window: $\beta = 8$. Calculate the filter coefficients using the Kaiser window: $h_{\text{kaiser}} = \text{firwin}(N, W_c, \text{window}=('kaiser', \beta))$. Plot the frequency response and impulse response for the Kaiser window. Display the plots.
The code

FIRWITHKASIERWINDOW.PY > ...

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.signal import firwin, freqz
4  # Define input specifications
5  wp = 0.375 * np.pi
6  ws = 0.5 * np.pi
7  # Calculate the order of the filter
8  N = int(np.ceil(8 * np.pi / (ws - wp)) + 1)
9  # Calculate the cutoff frequency
10  Wc = (wp + (ws - wp) / 2) / (0.5 * np.pi)
11  # Set the beta parameter for the Kaiser window
12  beta = 8
13  # Calculate the filter coefficients using the Kaiser window
14  h_kaiser = firwin(N, Wc, window=('kaiser', beta))
15  # Plot the frequency response
16  w, h = freqz(h_kaiser, worN=8000)
17  plt.figure()
18  plt.subplot(2, 1, 1)
19  plt.plot(0.5 * w / np.pi, np.abs(h), 'b')
20  plt.title('Frequency response of the Kaiser Window FIR Filter')
21  plt.xlabel('Normalized Frequency ( $\times \pi$  rad/sample)')
22  plt.ylabel('Magnitude')
23  plt.grid()
24  # Plot the impulse response
25  plt.subplot(2, 1, 2)
26  plt.stem(h_kaiser, use_line_collection=True)
27  plt.title('Impulse response of the Kaiser Window FIR Filter')
28  plt.xlabel('Samples')
29  plt.ylabel('Amplitude')
30  plt.grid()
31  plt.tight_layout()
32  plt.show()
```

The results (Screenshot)



Lab Task No 4:

Solution:

Brief description (3-5 lines)

Implementation of IIR Filters

Design three different IIR filters using the given specifications:

Filter order: 4

Cutoff frequency: 0.2 (normalized frequency)

Algorithm 1: Butterworth Filter

Use the butter function from SciPy to design a Butterworth filter with the given order and cutoff frequency.

Obtain the filter coefficients `b` and `a` using `butter`.

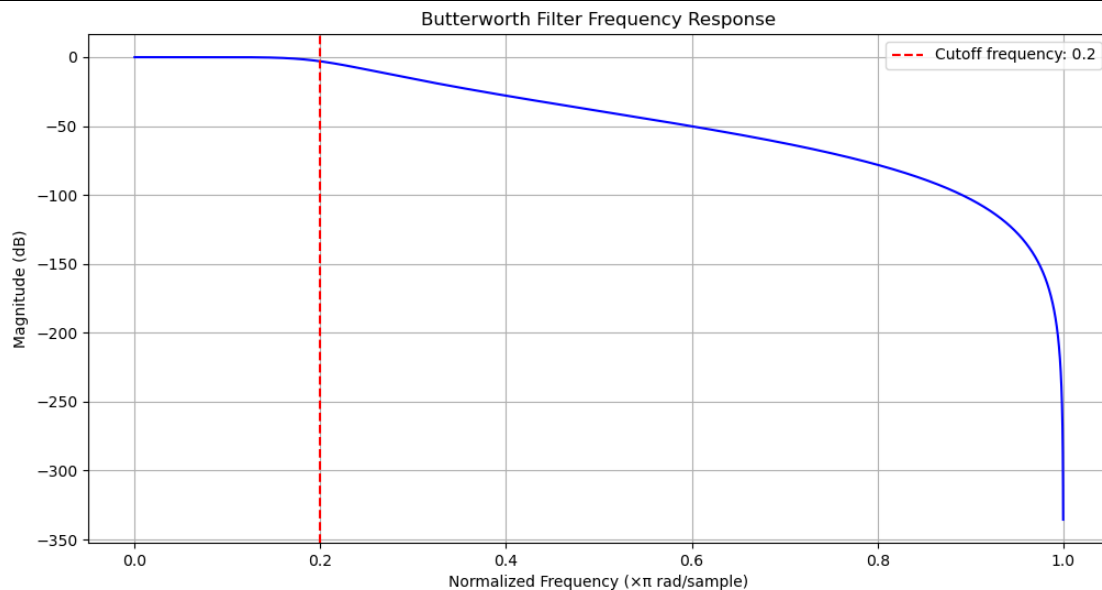
The code

```

Butterworth Filter.py > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.signal import butter, freqz
4  # Filter specifications
5  order = 4
6  cutoff = 0.2 # Normalized frequency (0 to 1, where 1 is the Nyquist frequency)
7  # Design Butterworth filter
8  b_butter, a_butter = butter(order, cutoff, btype='low', analog=False)
9  # Frequency response
10 w, h = freqz(b_butter, a_butter, worN=8000)
11 # Plot frequency response
12 plt.figure(figsize=(12, 6))
13 plt.plot(w / np.pi, 20 * np.log10(abs(h)), 'b')
14 plt.title('Butterworth Filter Frequency Response')
15 plt.xlabel('Normalized Frequency (xπ rad/sample)')
16 plt.ylabel('Magnitude (dB)')
17 plt.grid()
18 plt.axvline(cutoff, color='red', linestyle='--', label=f'Cutoff frequency: {cutoff}')
19 plt.legend()
20 # Display the plot
21 plt.show()
22 # Print filter coefficients
23 print("Butterworth filter coefficients:")
24 print("b (numerator):", b_butter)
25 print("a (denominator):", a_butter)

```

The results (Screenshot)



Lab Task No 5:

Solution:

Brief description (3-5 lines)

Algorithm 2: Chebyshev Type I Filter

Set the passband ripple to 1 dB.

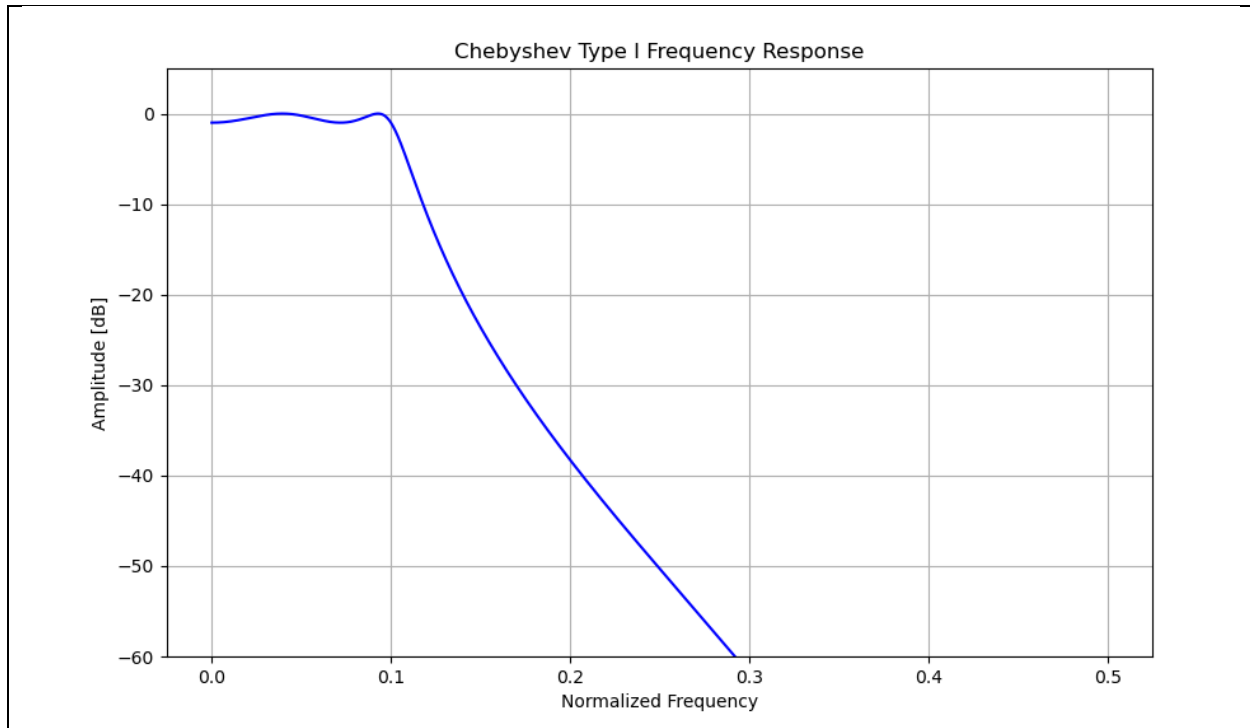
Use the `cheby1` function from SciPy to design a Chebyshev Type I filter with the given order, passband ripple, and cutoff frequency.

Obtain the filter coefficients `b_cheby1` and `a_cheby1`.

The code

```
Chebyshev Type I Filter.py > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.signal import cheby1, freqz
4  # Filter specifications
5  order = 4
6  ripple = 1 # in dB
7  cutoff_frequency = 0.2 # Normalized frequency (0 to 1, where 1 corresponds to Nyquist frequency)
8  # Design the Chebyshev Type I filter
9  b_cheby1, a_cheby1 = cheby1(order, ripple, cutoff_frequency, btype='low', analog=False)
10 # Compute the frequency response
11 w, h = freqz(b_cheby1, a_cheby1, worN=8000)
12 # Plot the frequency response
13 plt.figure(figsize=(10, 6))
14 plt.plot(0.5 * w / np.pi, 20 * np.log10(np.abs(h)), 'b')
15 plt.title('Chebyshev Type I Frequency Response')
16 plt.xlabel('Normalized Frequency')
17 plt.ylabel('Amplitude [dB]')
18 plt.ylim(-60, 5)
19 plt.grid()
20 plt.show()
```

The results (Screenshot)



Lab Task No 6:

Solution:

Brief description (3-5 lines)
Algorithm 3: Elliptic Filter Set the passband ripple to 1 dB and the stopband attenuation to 40 dB. Use the <code>ellip</code> function from SciPy to design an elliptic filter with the given order, passband ripple, stopband attenuation, and cutoff frequency. Obtain the filter coefficients <code>b_ellip</code> and <code>a_ellip</code>. Plot the magnitude response for each filter: Compute the frequency response of each filter using the <code>freqz</code> function from SciPy. Plot the magnitude response of the Butterworth filter using <code>plt.plot(w, np.abs(h_butter), label='Butterworth')</code>.


```

Plot the magnitude response of the Chebyshev Type I filter using
plt.plot(w, np.abs(h_cheby1),
label='Chebyshev Type I').
Plot the magnitude response of the elliptic filter using plt.plot(w,
np.abs(h_ellip),
label='Elliptic').
Set the title to 'Magnitude Response' using plt.title('Magnitude
Response').
Set the x-axis label to 'Frequency (radians)' using plt.xlabel('Frequency
(radians)').
Set the y-axis label to 'Magnitude' using plt.ylabel('Magnitude').
Display the legend using plt.legend().
Enable grid lines using plt.grid(True).
Plot the phase response for each filter:
Plot the phase response of the Butterworth filter using plt.plot(w,
np.angle(h_butter),
label='Butterworth').
Plot the phase response of the Chebyshev Type I filter using plt.plot(w,
np.angle(h_cheby1),
label='Chebyshev Type I').
Plot the phase response of the elliptic filter using plt.plot(w,
np.angle(h_ellip), label='Elliptic').
Set the title to 'Phase Response' using plt.title('Phase Response').
Set the x-axis label to 'Frequency (radians)' using plt.xlabel('Frequency
(radians)').
Set the y-axis label to 'Phase (radians)' using plt.ylabel('Phase
(radians)').
Display the legend using plt.legend().
Enable grid lines using plt.grid(True).
Display the plots using plt.show().

```

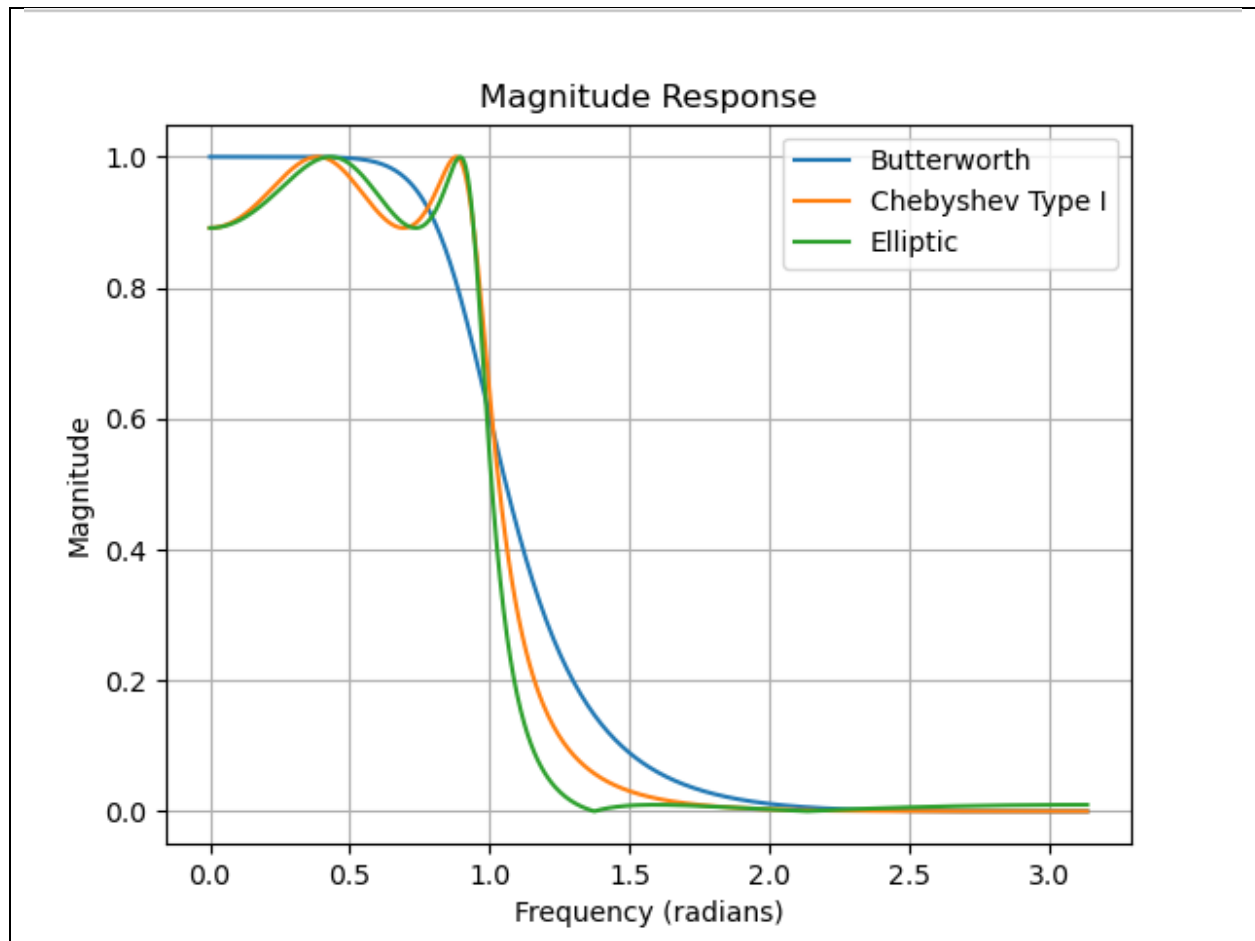
The code

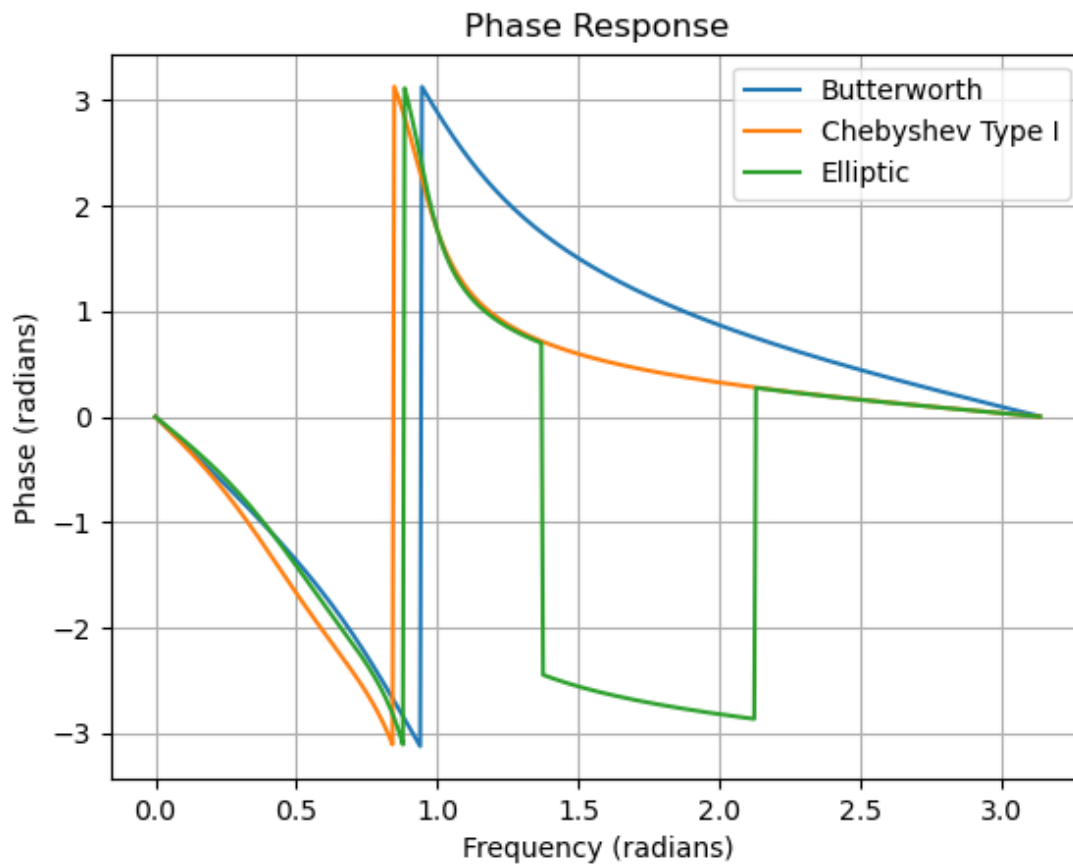
```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.signal import butter, cheby1, ellip, freqz
4  order = 4
5  cutoff = 0.3
6  rp = 1
7  rs = 40
8  b_butter, a_butter = butter(order, cutoff, btype='low', analog=False)
9  w, h_butter = freqz(b_butter, a_butter)
10 b_cheby1, a_cheby1 = cheby1(order, rp, cutoff, btype='low', analog=False)
11 w, h_cheby1 = freqz(b_cheby1, a_cheby1)
12 b_ellip, a_ellip = ellip(order, rp, rs, cutoff, btype='low', analog=False)
13 w, h_ellip = freqz(b_ellip, a_ellip)
14 plt.figure()
15 plt.plot(w, np.abs(h_butter), Label='Butterworth')
16 plt.plot(w, np.abs(h_cheby1), Label='Chebyshev Type I')
17 plt.plot(w, np.abs(h_ellip), Label='Elliptic')
18 plt.title('Magnitude Response')
19 plt.xlabel('Frequency (radians)')
20 plt.ylabel('Magnitude')
21 plt.legend()
22 plt.grid(True)
23 plt.figure()
24 plt.plot(w, np.angle(h_butter), Label='Butterworth')
25 plt.plot(w, np.angle(h_cheby1), Label='Chebyshev Type I')
26 plt.plot(w, np.angle(h_ellip), Label='Elliptic')
27 plt.title('Phase Response')
28 plt.xlabel('Frequency (radians)')
29 plt.ylabel('Phase (radians)')
30 plt.legend()
31 plt.grid(True)
32 plt.show()

```

The results (Screenshot)





Conclusion

In conclusion, digital signal processing (DSP) employs filters like FIR (Finite Impulse Response) and IIR (Infinite Impulse Response) to manipulate signal characteristics. FIR filters offer stability and a linear phase response, while IIR filters provide computational efficiency with potential for instability.